CircuitPilot 操作指导说明书

安装 · 本地部署 · 嘉立创EDA · 桥接 · MCP 边界 · 基础配置

CircuitPilot 项目

2026-10-26

CircuitPilot 操作指导说明书

目录

1. 5 分钟快速开始
2. 安装 CircuitPilot
3. 首次配置
4. 安装嘉立创EDA专业版
5. 桥接：让 CircuitPilot 连接到 EDA
6. MCP 设置说明（重要边界）
7. 日常操作流程
8. 安全设置与红线
9. 故障排查
10. 命令、端口、目录速查
11. 验收清单

附录 A：术语

附录 B：相关文档

附录 C：版本与支持

CircuitPilot 操作指导说明书

适用版本： CircuitPilot v0.2.1（macOS）
文档日期： 2026-10-26
适用范围： 桌面安装包用户、源码本地部署用户、需要连接嘉立创EDA专业版的用户

> **先读这一条：** CircuitPilot 是实验性软件，不是"一键投产"工具。它不会自动下单、不会自动付款，也不会伪造 DRC/ERC/DFM/签核结果。任何写入都必须经过人工批准和回读验证；证据不足时，放行门禁会保持阻断，这是设计行为，不是故障。

目录

1. 5 分钟快速开始
2. 安装 CircuitPilot
3. 首次配置
4. 安装嘉立创EDA专业版
5. 桥接：让 CircuitPilot 连接到 EDA
6. MCP 设置说明（重要边界）
7. 日常操作流程
8. 安全设置与红线
9. 故障排查
10. 命令、端口、目录速查
11. 验收清单

---

1. 5 分钟快速开始

如果你已经拿到 macOS 安装包，按下面顺序做：

1. 打开 GitHub Releases：https://github.com/206758633-sudo/CircuitPilot-AI-PCB-CONTROL/releases
2. 下载 `CircuitPilot_0.2.1_aarch64.dmg`（Apple Silicon）或对应架构的安装包。
3. 对照同页 `SHA256SUMS.txt` 校验下载文件（见 2.1.2）。
4. 打开 DMG，把 `CircuitPilot.app` 拖进 `/Applications`。
5. 首次启动：在 Finder 的"应用程序"里 **右键 → 打开**（安装包未做 Apple 公证）。

6. 进入应用后，先在 **系统设置 → Provider** 选择 `Demo 本地模拟`，点 **测试连接**。
7. 点 **全局暂停所有写入**（演示/首次使用建议保持开启）。
8. 在 **项目与工程资料** 新建一个测试项目，先用 Demo 数据走一遍界面。

完成后，再按第 4、5 章安装嘉立创EDA并配置桥接。

---

2. 安装 CircuitPilot

2.1 macOS 安装包（推荐普通用户）

2.1.1 系统要求

项目	要求
操作系统	macOS 12 Monterey 或更新
芯片	Apple Silicon（arm64）或 Intel（x86_64），以 Release 页面提供的架构为准
磁盘	至少 1 GB 可用空间（另加项目/证据空间）
网络	仅在使用云端 AI Provider 时需要
Node.js	**安装包用户不需要**，应用内置 Node Runtime

2.1.2 下载与校验

1. 打开 Releases 页面，下载 `.dmg` 和 `SHA256SUMS.txt`。
2. 打开"终端"，执行：

```
cd ~/Downloads
shasum -a 256 CircuitPilot_0.2.1_aarch64.dmg
```

3. 把输出与 `SHA256SUMS.txt` 中对应行对比。**不一致就不要安装**，重新下载。

2.1.3 安装

1. 双击 `.dmg`。
2. 把 `CircuitPilot.app` 拖到 `Applications` 文件夹。
3. 推出磁盘映像。
4. 在"应用程序"中找到 CircuitPilot，**右键 → 打开**。

如果 macOS 提示"无法验证开发者"或"已损坏"：

```
# 仅在你确认文件来自本项目官方 Release 时执行
xattr -dr com.apple.quarantine /Applications/CircuitPilot.app
open /Applications/CircuitPilot.app
```

> 安全提示：当前 Release 构建是 **ad-hoc 签名、未公证**。Gatekeeper 提示是预期现象。校验 SHA256 后手动放行，风险由你自行承担；生产分发应配置 Apple Developer ID + 公证。

2.1.4 首次启动会创建什么

内容	默认位置
应用状态（项目、trace、设置）	`~/Library/Application Support/com.circuitpilot.desktop/state`
证据文件	`~/Library/Application Support/com.circuitpilot.desktop/state/evidence`
Runtime 日志	`~/Library/Logs/com.circuitpilot.desktop/runtime.log`
Provider 密钥	macOS 钥匙串（不以明文写入状态目录）

这些数据不会上传到本项目仓库或云端。

2.1.5 卸载

1. 退出 CircuitPilot。
2. 删除 `/Applications/CircuitPilot.app`。
3. 如需彻底清除数据（**会丢失项目/证据，先备份**）：

```
rm -rf ~/Library/Application\ Support/com.circuitpilot.desktop
rm -rf ~/Library/Logs/com.circuitpilot.desktop
```

2.1.6 升级

用新版本 `.app` 覆盖 `/Applications/CircuitPilot.app` 即可，状态目录默认保留。升级前建议备份状态目录：

```
cp -R ~/Library/Application\ Support/com.circuitpilot.desktop/state \
      ~/Desktop/circuitpilot-state-backup-$(date +%Y%m%d)
```

2.2 源码本地部署（开发者 / 需要改代码）

2.2.1 前置条件

工具	版本/说明
macOS	12+
Node.js	22 或更高
npm	随 Node 安装
Rust	stable 工具链（`rustc`、`cargo`）
Xcode Command Line Tools	`xcode-select --install`
Git	任意近期版本

检查：

```
node -v
npm -v
rustc -vV
xcode-select -p
```

2.2.2 获取代码并安装依赖

```
git clone https://github.com/206758633-sudo/CircuitPilot-AI-PCB-CONTROL.git
cd CircuitPilot-AI-PCB-CONTROL
npm ci
```

2.2.3 启动 Web 控制台 + 本地 Runtime

```
npm run dev:full
```

- 控制台：http://127.0.0.1:4177
- Runtime：http://127.0.0.1:4178（外部请求可能因令牌保护返回 401，属预期）
- `Ctrl+C` 会同时停止 Vite 和 Runtime

只启动前端（Runtime 需另开）：

```
npm run dev
npm run dev:runtime
```

2.2.4 启动桌面开发版

```
npm run desktop:dev
```

首次会编译 Rust，可能需要几分钟。

2.2.5 构建 macOS 应用

```
npm run desktop:app
# 产物：src-tauri/target/release/bundle/macos/CircuitPilot.app
```

构建 DMG（不执行公证门禁，适合本地/未签名分发）：

```
npm run sidecar:stage
npx tauri build --bundles app,dmg
# 产物：src-tauri/target/release/bundle/dmg/*.dmg
```

> `npm run desktop:release` 会先执行 Apple Developer ID/公证检查；没有证书时会 **fail-closed**，这是有意的发布门禁。未签名测试包请用上面的 `tauri build` 命令。

2.2.6 常用开发命令

命令	用途
`npm run typecheck`	TypeScript 检查
`npx vitest run`	前端单元测试
`npm run test:runtime`	Runtime/Node 测试
`npm run test:e2e`	Playwright E2E
`npm run build`	Web 生产构建
`npm run test:coverage`	覆盖率门禁

2.2.7 开发用环境变量

复制 `.env.example` 为 `.env`（不要提交 `.env`）。常用项：

变量	默认	用途
`CIRCUITPILOT_STATE_DIR`	应用数据目录/`artifacts/state`	状态目录
`CIRCUITPILOT_EVIDENCE_DIR`	`<state>/evidence`	证据目录
`CIRCUITPILOT_EDA_CDP_PORT`	`9222`	嘉立创EDA CDP 端口
`CIRCUITPILOT_BRIDGE_PORT`	`49620`	自动桥接进程的基础端口（探测 10 个）
`CIRCUITPILOT_RUNTIME_PORT`	`4178`	本地 Runtime 端口
`CIRCUITPILOT_GATE_DRY_RUN`	未设置	强制门禁 dry-run

---

3. 首次配置

3.1 认识主界面

导航	作用
概览	项目健康度、待处理问题、关键状态
AI 设计	需求对话、需求约束、布局微调、资料/选型
EDA 执行	目标身份、执行计划、审批、写入与回读
自动化	受控配方/任务（高风险，默认谨慎）
验证与交付	ERC/DRC、引脚/网络审计、制造检查、放行门禁
版本与问题	版本、trace、问题中心
系统设置	Provider、EDA 连接、安全开关、诊断
项目与工程资料	新建/切换项目、工程目录

首次使用，先保持 **Demo 模式** 和 **全局暂停写入**，把界面和流程走一遍。

3.2 配置 AI Provider

进入 **系统设置 → Provider**，按你的实际情况选择：

Provider	适用场景	配置方式
`Demo 本地模拟`	只看界面/流程，不调用真实模型	直接选择，点"测试连接"
`Codex CLI`	已在本机安装并登录 Codex CLI	选择后按提示启动官方登录；应用只读取本地登录态
`Claude Code`	已在本机安装并登录 Claude Code	同上；注意仅用于本机个人工作流
OpenAI / Anthropic API	使用官方 API Key	在设置中粘贴 Key → "安全保存"（写入钥匙串）
自定义兼容端点	OpenAI/Anthropic 兼容 API	填 HTTPS 地址（本机回环可用 HTTP）+ Key

步骤：

1. 选择 Provider。
2. 点 **测试连接**。
3. 看到"已就绪/认证有效/模型数量"等结果后，再点 **保存设置**。
4. 如果使用 API Key：保存后不要把 Key 写进任何文件、截图或 issue。

> 安全边界：Provider Key 只保存在 macOS 钥匙串；CircuitPilot 不会把 Key 写进项目状态或日志。若你要把日志发给别人，请先使用应用内诊断脱敏功能。

3.3 新建项目

进入 **项目与工程资料 → 新建项目**：

字段	建议
项目名称	真实、可识别，例如 `Demo Sensor Board`
项目代号	短代号，例如 `DEMO-A01`
项目描述	目标、约束、版本
本地工作目录	你本机用于保存工程资料的目录
嘉立创EDA工程路径	EDA 工程文件/目录的本地路径（仅记录，不会自动打开）

创建后，项目会出现在项目切换器中。正式写入前，务必确认当前项目、EDA 工程、原理图/PCB 文档三者是同一个目标。

3.4 基础安全设置

建议首次使用保持：

- **全局暂停所有写入：开启**

- **Demo 模式：开启**（未连接真实 EDA 时）
- **自动重开工程：关闭**（除非你明确需要）
- **审批模式：逐次人工批准**

确认位置以当前版本界面为准，通常在 **系统设置 → 安全/自动化** 区域。

---

4. 安装嘉立创EDA专业版

> CircuitPilot 的 EDA 写入依赖你本机已安装、已登录的嘉立创EDA**专业版**。CircuitPilot 不分发、不修改 EDA 软件本身。

4.1 下载

1. 打开嘉立创EDA官网：https://lceda.cn/
2. 进入"客户端下载/专业版"页面，选择 macOS 版本。
3. 下载 `.dmg`，打开后拖入 `Applications`。
4. 首次打开如被 Gatekeeper 拦截，按 EDA 官方说明放行。

4.2 注册与登录

1. 启动嘉立创EDA专业版。
2. 注册/登录嘉立创账号（企业/团队账号也可）。
3. 确认能正常打开"工作区/工程"。
4. 建议先在 EDA 中手动新建一个空白测试工程，确认保存、关闭、重开都正常。

4.3 工程准备

1. 新建或打开目标工程。
2. 确认工程内至少有：原理图文档、PCB 文档。
3. 保存一次，确保工程文件落在本机磁盘（而不是只在浏览器缓存/云端未同步状态）。
4. 记住工程名称、原理图名称、PCB 名称；CircuitPilot 执行前会要求目标身份匹配。
5. **在写入验证完成前，不要升级 EDA 大版本**，否则 DOM/API 行为可能变化。

4.4 与 CircuitPilot 的版本关系

- CircuitPilot 通过 EDA 的本地调试接口（CDP）读取/操作当前编辑器。
- EDA 升级后如果界面结构变化，部分 GUI 兜底操作可能失效；此时应以应用内"EDA 检测/能力探测"结果为准。

- 不要把 EDA 的云工程当作唯一副本；正式操作前保留本地备份/版本。

---

5. 桥接：让 CircuitPilot 连接到 EDA

5.1 桥接原理

CircuitPilot 的"桥接"不是 MCP，而是：

```
CircuitPilot 桌面应用
  ├─ 本地 Runtime（127.0.0.1:4178，带启动令牌）
  │    ├─ 嘉立创EDA 桥：Chrome DevTools Protocol（CDP，默认 127.0.0.1:9222）
  │    └─ API Gateway 桥：CircuitPilot 自动启动，默认端口 49620 起
  └─ React 控制台（只通过 Runtime 发请求）
```

- **CDP 桥**：CircuitPilot 像调试器一样连接 EDA 的 Chromium 内核，执行受控的读取/点击/输入/API 调用。
- **API Gateway 桥（端口 49620–49629）**：CircuitPilot 启动时会自动拉起本地桥接守护进程；EDA 内需要安装并启用官方扩展 `run-api-gateway`（**API Gateway**），并在扩展设置里打开"外部交互权限"。没有这个扩展时，EDA 会显示未连接。
- 桥接守护进程由 CircuitPilot 自动启动，不需要用户手动运行 `node bridge-server.mjs`。
- 两者都只在本机回环地址工作，不经过 CircuitPilot 的云服务。

5.2 启动带调试端口的嘉立创EDA（macOS）

方式 0（推荐，CircuitPilot 内一键启动）：

1. 打开 CircuitPilot → **EDA 执行**。
2. 点击顶部的 **"启动EDA"** 按钮。
3. 应用会用调试端口 9222 启动本机嘉立创EDA；如果 EDA 已在调试模式运行，会直接提示已就绪。
4. 回到"连接与环境"点 **"重连 / 运行全量检查"**。

> 这个按钮只启动 EDA，不会修改工程；仍然需要你在 EDA 里打开目标工程，并确认已安装 `run-api-gateway` 扩展。

方式 A/B（终端，备用）：

先完全退出 EDA（`Cmd+Q`，确认 Dock 中没有运行实例），再执行：

```
# 方式 A：使用仓库脚本（推荐）
./scripts/launch-eda.sh 9222

# 方式 B：手动启动
env -u ELECTRON_RUN_AS_NODE \
  "/Applications/嘉立创EDA(专业版).app/Contents/MacOS/LCEDA-Pro" \
  --remote-debugging-port=9222 \
  > /tmp/lceda-run.log 2>&1 &
```

说明：

- `env -u ELECTRON_RUN_AS_NODE` 用于清除某些开发环境设置的变量；否则 Electron 可能以 Node 模式启动，表现为"双击没反应/秒退"。
- 应用名可能是 `嘉立创EDA(专业版).app`，以你 `/Applications` 中的实际名称为准。
- 端口默认 `9222`；如被占用可换 `9223`，并在 CircuitPilot 设置/环境变量 `CIRCUITPILOT_EDA_CDP_PORT` 中同步。

5.3 验证 CDP 是否就绪

```
curl -s http://127.0.0.1:9222/json/version
curl -s http://127.0.0.1:9222/json/list | head
```

期望：

- `/json/version` 返回包含 `Browser`、`webSocketDebuggerUrl` 的 JSON；
- `/json/list` 中至少有一个 `"type": "page"` 的 EDA 页面；
- 如果连接被拒绝，说明 EDA 没有以调试端口启动。

端口占用排查：

```
lsof -nP -iTCP:9222 -sTCP:LISTEN
```

5.3.1 安装 API Gateway 扩展（新用户必做一次）

1. 在嘉立创EDA专业版中打开 **扩展 / 扩展广场**。
2. 搜索并安装官方扩展 **`run-api-gateway`**（显示名通常为"API Gateway"）。
3. 在扩展设置中开启 **外部交互权限 / External Interaction Permission**。
4. 重新加载/重启 EDA，让扩展自动连接本地桥接服务（端口 49620–49629）。
5. CircuitPilot 的"连接与环境"里应能看到 `EasyEDA Bridge` 通过。

> 如果扩展广场里找不到，请确认 EDA 版本与官方扩展市场可用；CircuitPilot 不分发这个官方扩展，也不会绕过 EDA 的权限设置。

5.4 在 CircuitPilot 中确认连接

1. 打开 CircuitPilot。
2. 进入 **系统设置 → EDA 连接**（或界面上的 EDA 状态卡片）。
3. 点 **刷新本机状态/检测 EDA**。
4. 确认显示：EDA 已连接、当前工程/文档可读。
5. 进入 **EDA 执行**，先执行"读取当前目标/只读检查"，确认工程名、原理图、PCB 与你在 EDA 中打开的一致。

如果显示未连接，按顺序检查：

- EDA 是否用 `--remote-debugging-port=9222` 启动；
- 是否有多个 EDA 实例/旧进程占用；
- CircuitPilot 的端口设置是否与启动参数一致；
- 防火墙/安全软件是否阻止本机回环；
- 查看 `/tmp/lceda-run.log` 与 Runtime 日志。

5.5 第一次连接后的只读验证（强烈建议）

在正式写入前，至少完成：

1. 读取当前工程信息。
2. 读取当前原理图/PCB 文档身份。
3. 读取器件数量、网络数量。
4. 执行一次只读 DRC/ERC（如果当前版本支持）。
5. 截图/记录目标身份（工程名 + 文档名 + 时间）。

只有当只读结果与 EDA 界面一致，才进入写入流程。

5.6 写入流程与回滚

1. 在 AI 设计页形成明确计划（改什么、改哪些位号/网络）。
2. 在 EDA 执行页核对目标身份和计划哈希。
3. 人工批准（逐次，不要批量放行）。
4. CircuitPilot 写入 → **回读** → 运行 ERC/DRC。
5. 失败时使用快照/回滚；不要用"再点一次"掩盖半写入。
6. 把结果写入证据；失败也是一条证据。

5.7 桥接常见问题

- 显示 `BRIDGE_NOT_FOUND`：CircuitPilot 自动启动的桥进程没起来，或 49620–49629 被占用；先完全退出 EDA 和 CircuitPilot，再重开应用。

- 显示 `edaConnected: false`：桥在，但 EDA 没有连接；检查 `run-api-gateway` 是否安装、是否启用“外部交互权限”、是否重启过 EDA。
- 显示多个窗口：在高级控制的窗口选择器里人工选定目标，不要同时操作多个工程。
- 端口被占用：用 `lsof -nP -iTCP:49620-49629 -sTCP:LISTEN` 查看；关闭旧桥/旧 EDA 实例后再试。
- 不要把桥端口暴露到局域网或公网；桥只应监听 `127.0.0.1`。

---

6. MCP 设置说明（重要边界）

6.1 结论

CircuitPilot v0.2.1 当前没有 MCP Server，也没有 MCP Client，不读取 `mcp.json`。
因此：

- 不需要在 Claude Desktop、Cursor、Claude Code 等客户端里添加一个”CircuitPilot MCP”。
- 仓库和安装包中都没有可配置的 MCP 服务地址/命令。
- 网上如果出现”CircuitPilot MCP 配置”，不是本仓库提供的，请勿直接照抄。

6.2 CircuitPilot 的”桥接”与 MCP 的区别

项目	CircuitPilot 当前桥接	MCP
协议	本机 CDP + 本地 HTTP/WebSocket	Model Context Protocol（JSON-RPC）
面向	CircuitPilot Runtime ↔ EDA 编辑器	AI 客户端 ↔ 工具/数据源
默认端口	9222（CDP）、49620+（EasyEDA 可选）、4178（Runtime）	由各 MCP 客户端配置
认证	每次启动注入令牌 + 回环限制	由 MCP 客户端/服务端约定
当前状态	已实现	**未实现**

6.3 那你应该怎么让 AI 参与设计?

使用 CircuitPilot **内置的 Provider + AI 设计页**：

1. 系统设置 → Provider 选择并测试可用模型；
2. AI 设计页描述需求；
3. 生成/编辑结构化计划；
4. EDA 执行页人工批准；
5. 验证与交付页收集证据。

这是当前版本支持的、有审批与证据边界的路径。

6.4 如果你要的是"外部 AI 客户端通过 MCP 控制 CircuitPilot"

当前版本不支持。实现它需要额外开发，至少包括：

- 新增 MCP Server（stdio 或本地 HTTP）；
- 复用 Runtime 的启动令牌与审批账本；
- 只读工具白名单、写入工具强制人工确认；
- 审计日志与证据绑定；
- 防止 MCP 客户端绕过 `全局暂停写入`。

在这些完成前，**不要**用第三方 MCP 工具直接操作 EDA；那会绕过 CircuitPilot 的审批/回读/证据机制。

6.5 第三方 EDA MCP

如果你使用的是某个第三方"EDA MCP Server"，它与 CircuitPilot 是并行关系，不是 CircuitPilot 的桥接：

- 不要把 MCP 端口填到 CircuitPilot 的 CDP 端口；
- 不要让两个工具同时写同一个工程；
- 第三方 MCP 的权限/证据边界由它自己负责，CircuitPilot 不会为其背书。

---

7. 日常操作流程

7.1 需求 → 计划

1. 选择项目。
2. 打开 **AI 设计**，描述需求（电源、接口、MCU、尺寸、连接器方向、灯/按键等）。
3. 检查 AI 输出的"设计事实"和"待确认问题"；不确定的地方必须人工确认。
4. 生成/编辑计划，确认位号、网络、封装、约束。

7.2 执行 → 回读

1. 打开 **EDA 执行**。
2. 核对目标身份（工程/文档/UUID/名称）。
3. 核对计划哈希与变更范围。
4. 人工批准。
5. 观察执行日志、回读结果、ERC/DRC 状态。
6. 失败就停在失败处，保留现场，不要连续重试掩盖问题。

7.3 验证 → 放行门禁

1. 打开 **验证与交付**。
2. 收集 ERC、DRC、引脚/网络审计、仿真（可选）、制造检查（如提供 BOM/CPL/Gerber/STEP）。
3. DFM 必须由你本人在板厂门户完成；CircuitPilot 不会伪造在线 DFM。
4. 最终签核绑定到 manifest 哈希。
5. 门禁显示"阻断"时，逐条看 blocker；不要为了变绿而删除证据。

7.4 Trace / 版本 / 问题

- **版本与问题** 页记录变更、审计、问题中心。
- 重要操作后导出/备份状态目录与证据。
- 出现"孤儿路由/未接线"提示时，说明功能尚未接 UI，不代表可以绕过审批。

---

8. 安全设置与红线

8.1 必须遵守

- 写入前确认目标身份；不确定就停。
- 保持"全局暂停写入"直到你准备好。
- 逐次批准，不使用批量放行。
- 保留快照/备份；失败先回滚/恢复。
- 不把 API Key、客户文件、证据包上传到公开 issue。
- 不把 Runtime 端口暴露到公网/局域网。
- 不伪造 DRC/ERC/DFM/签核结果。

8.2 CircuitPilot 永远不会做

- 自动下单、自动付款、自动提交板厂。
- 在没有证据时显示"通过"。
- 把 demo/样例/模拟数据当作真实验收。
- 绕过人工审批直接写板。

8.3 令牌与 401

桌面版每次启动生成随机令牌，通过 sidecar stdin 与 WebView 注入。外部进程直接访问 `http://127.0.0.1:4178/api/*` 返回 `401` 是**预期安全行为**，不是 Runtime 故障。不要尝试关闭令牌校验。

9. 故障排查

9.1 应用打不开 / 提示已损坏

- 确认下载来源和 SHA256；
- 执行 `xattr -dr com.apple.quarantine /Applications/CircuitPilot.app`；
- 右键 → 打开；
- 仍失败：查看 Console.app 中 `CircuitPilot` 日志，确认 macOS 版本/架构匹配。

9.2 界面能打开，但 Runtime 未就绪

- 桌面版：完全退出后重开（让 sidecar 重启）；
- 开发版：确认 `npm run dev:full` 正在运行，4178 未被占用；
- 查看 `~/Library/Logs/com.circuitpilot.desktop/runtime.log`；
- 端口：`lsof -nP -iTCP:4178 -sTCP:LISTEN`。

9.3 EDA 检测不到

按顺序：

1. EDA 是否带 `--remote-debugging-port=9222` 启动?
2. `curl -s http://127.0.0.1:9222/json/version` 是否有返回?
3. EDA 是否安装了官方 **`run-api-gateway`** 扩展，并开启“外部交互权限”?
4. CircuitPilot 是否自动启动了桥接进程：`curl -s http://127.0.0.1:49620/health` 是否返回 `easyeda-bridge`?
5. 是否有旧的 EDA 进程/多个实例?
6. CircuitPilot 端口设置是否一致?
7. `/tmp/lceda-run.log` 是否有 Electron 错误?
8. 重启 EDA 后再点“刷新本机状态”。

9.4 Provider 测试失败

- Demo：应当直接成功；
- CLI：确认本机已安装并登录，终端能运行对应命令；
- API：检查 Key、Base URL、网络、代理；钥匙串权限；
- 自定义端点：必须是 HTTPS（本机回环可 HTTP），不要带用户名密码。

9.5 写入被拒绝

常见原因（都是保护）：

- 目标身份不匹配；
- 计划不完整/哈希未批准；
- 全局暂停写入开启；
- 缺少快照；
- 审批已过期/被撤销；
- 当前是 Demo 模式。

9.6 放行门禁一直阻断

这不是 bug。按 blocker 列表补证据：

- ERC/DRC 是否针对当前文档；
- BOM/CPL 位号是否匹配；
- 是否缺少 DFM/签核；
- 是否 demo 数据；
- 证据是否过期/哈希不匹配。

9.7 磁盘/权限

- 确保状态目录可写；
- 备份后清理旧证据；
- 不要把状态目录放在 iCloud/同步盘上做高频写入。

---

10. 命令、端口、目录速查

10.1 端口

端口	用途	默认
4177	Vite 控制台（开发）	开发
4178	本地 Runtime	是
9222	嘉立创EDA CDP	是
49620–49629	本地 API Gateway 桥（自动启动）	探测

10.2 目录

路径	内容
`~/Library/Application Support/com.circuitpilot.desktop/state`	状态
`.../state/evidence`	证据
`~/Library/Logs/com.circuitpilot.desktop/runtime.log`	日志
`src-tauri/target/release/bundle/macos/`	本地构建 `.app`
`src-tauri/target/release/bundle/dmg/`	本地构建 `.dmg`

10.3 常用命令

```
# 源码启动
npm ci
npm run dev:full

# 桌面开发
npm run desktop:dev

# 构建 app
npm run desktop:app

# 构建 dmg（未签名）
npm run sidecar:stage
npx tauri build --bundles app,dmg

# 启动 EDA CDP
./scripts/launch-eda.sh 9222
curl -s http://127.0.0.1:9222/json/version

# 校验下载包
shasum -a 256 CircuitPilot_0.2.1_aarch64.dmg
```

10.4 重要环境变量

见 配置文档 与仓库根目录 `.env.example`。

---

11. 验收清单

11.1 安装验收

- [ ] `.dmg` SHA256 与 `SHA256SUMS.txt` 一致

- [ ] 应用能启动，界面正常
- [ ] 系统设置能打开并保存
- [ ] Demo Provider 测试连接成功
- [ ] 状态目录/日志路径存在且可写

11.2 EDA 桥接验收

- [ ] EDA 以 `--remote-debugging-port=9222` 启动
- [ ] `curl /json/version` 有返回
- [ ] CircuitPilot 检测到 EDA
- [ ] 能读取当前工程/文档身份
- [ ] 只读检查结果与 EDA 界面一致
- [ ] 未开启写入时，写入请求被拒绝

11.3 安全验收

- [ ] 全局暂停写入可用
- [ ] 写入需要人工批准
- [ ] 失败会回读/回滚并留证据
- [ ] 门禁缺证据时阻断
- [ ] 外部直接访问 Runtime 返回 401
- [ ] 没有把 Key/客户文件/证据提交到 git

11.4 文档验收

- [ ] 已阅读本说明书第 6 章（MCP 边界）
- [ ] 知道当前版本没有 MCP，不要配置假 MCP
- [ ] 知道桥接是 CDP/本地 HTTP，不是 MCP

附录 A：术语

术语	说明
Runtime	本机 Node 服务，负责状态、证据、EDA 适配、门禁
CDP	Chrome DevTools Protocol，CircuitPilot 连接 EDA 的调试协议
Provider	AI 模型来源（CLI/API/自定义端点/Demo）
计划哈希	对执行计划的摘要，用于人工审批绑定
回读	写入后重新读取实际对象，确认结果
放行门禁	对证据做 fail-closed 判定的策略层
MCP	Model Context Protocol；当前版本未实现

附录 B：相关文档

- 安装说明
- 配置说明
- EDA 连接
- 安全模型
- 能力与限制
- 常见问题

附录 C：版本与支持

- 版本：v0.2.1
- 仓库：https://github.com/206758633-sudo/CircuitPilot-AI-PCB-CONTROL
- 问题反馈：GitHub Issues
- 安全漏洞：GitHub Security Advisories（不要开公开 issue）

> 本文档随版本更新；如界面文字与本文不一致，以当前版本界面为准，并欢迎提交文档修正。